BCA Sem - 3
Programing using Java

Unit - 4
Multi Threading



Introduction

In modern Operating Systems like Windows, MacOS we know that they can execute several programs simultaneously. This is known as **multitasking.** In system's terminology it is called as **multithreading.**

In other words MultiThreading is a concept where the program (process) is divided into two or more subprograms (processes), which can be implemented at the same time in parallel.

Introduction

For example : One subprogram is displaying an animation on the screen while another subprogram will build the next animation to be displayed. This is something like dividing a task into subtasks and assigning them to different people for execution independently and simultaneously.

In most of our computers we have single processor and therefore in reality the processor is doing one thing at a time. But the processor switches between the processes so fast that it appears that they are being done simultaneously.

Introduction

A thread is similar to a program that has a single flow of control. It has beginning, a body and an end. So in other words we can say that all our programs till to date are single-threaded programs.

A unique feature of java is its support for multithreading. A program that contains multiple flows of control is known as multithreaded program.

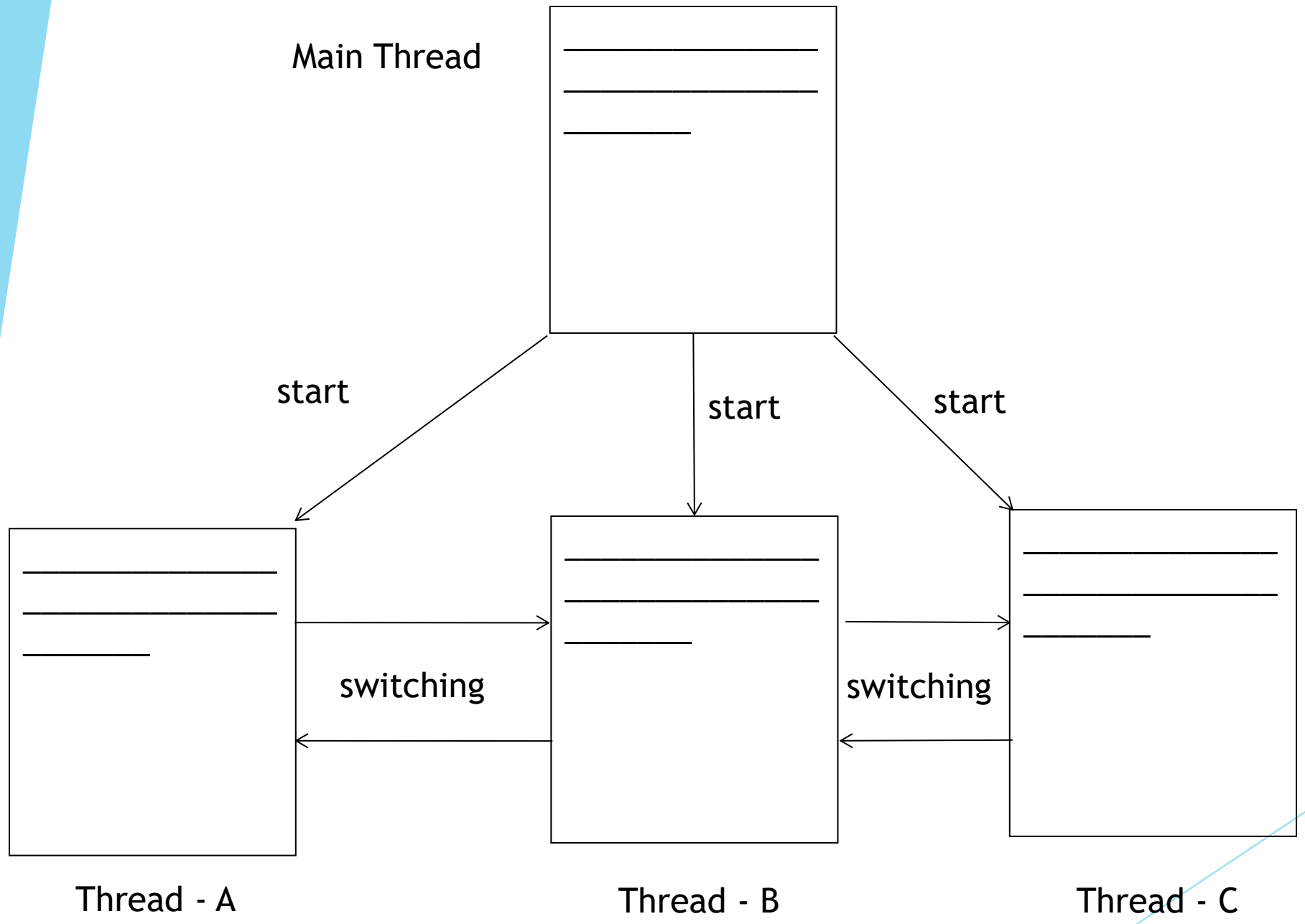
Remember that : The Java Virtual Machine (JVM) manages these and schedules them for execution.

Introduction

A thread is a sequence of execution within a process and we may think thread as a "lightweight" process.

There may be several threads in one process and Java Virtual Machine (JVM) will manage the schedules of execution of them.

The time needed to perform context switching from one thread to another is quite less than that required for performing such a change between processes - it is a benefit of Threading.


Main Thread
start
start
start
switching
switching
Thread - A
Thread - B
Thread - C

Creation of New Thread

Java provides two ways to create a new thread.

(1)By Extending the Thread class which is available in package java.lang.Thread

e.g.

```
class myThread extends Thread
{
        ……..
        ……..
}
```

1. Creation of New Thread

```
class ex161 extends Thread
{
    public void run()// must be public
    {
        System.out.println("thread is running...");
    }
    public static void main(String args[])
    {
        ex161 t1=new ex161();
        t1.start();
    }
}
```

2.Creation of New Thread

```
import java.lang.*;
class a extends Thread
{
	public void run()
	{
		for(int i=1;i<=5;i++)
		{
			System.out.println("From Thread A : i = "+i);
		}
		System.out.println("Exit from Thread A");
	}
}
class b extends Thread
{
	public void run()
	{
		for(int j=1;j<=5;j++)
		{
			System.out.println("From Thread B : j = "+j);
		}
		System.out.println("Exit from Thread B");
	}
}
```

2.Creation of New Thread

```
class c extends Thread
{
	public void run()
	{
		for(int k=1;k<=5;k++)
		{
			System.out.println("From Thread C : k = "+k);
		}
		System.out.println("Exit from Thread C");
	}
}
class ex163
{
	public static void main(String args[])
	{
		a a1=new a();
		a1.start();
		b b1=new b();
		b1.start();
		c c1=new c();
		c1.start();
	}
}
```

Creation of New Thread

Java provides two ways to create a new thread.

(2) Implement the Runnable interface from the package java.lang.Runnable

e.g.

```
class myThread implements Runnable
{
        ……..
        ……..
}
```

Creation of New Thread

```
class ex162 implements Runnable
{
    public void run()
    {
        System.out.println("thread is running...");
    }

    public static void main(String args[])
    {
        ex162 m1=new ex162();
        Thread t1 =new Thread(m1);
        t1.start();
    }
}
```

Runnable Interface

we know that threads can be created in two ways :

(1) By extending Thread class

(2) By implementing Runnable Interface.

Runnable Interface will work same as extending thread class. But it is useful when we need to extend some other class at that time this implementation will be much useful.

Runnable Interface

```
import java.lang.*;
class runna implements Runnable
{
	public void run()
	{
		for(int i=1;i<=5;i++)
		{
			System.out.println("From Thread A : i = "+i);
		}
		System.out.println("Exit From Thread A");
	}
}
class runnb implements Runnable
{
	public void run()
	{
		for(int j=1;j<=5;j++)
		{
			System.out.println("From Thread B : j = "+j);
		}
		System.out.println("Exit From Thread B");
	}
}
```

Runnable Interface

```
class runnc implements Runnable
{
	public void run()
	{
		for(int k=1;k<=5;k++)
		{
			System.out.println("From
Thread C : k = "+k);
		}
		System.out.println("Exit	From
Thread C");
	}
}
```

```
class ex26
{
	public static void main(String args[])
	{
		runna ra = new runna();
		runnb rb = new runnb();
		runnc rc = new runnc();

		Thread ta = new Thread(ra);
		System.out.println("Thread A Started");
		ta.start();

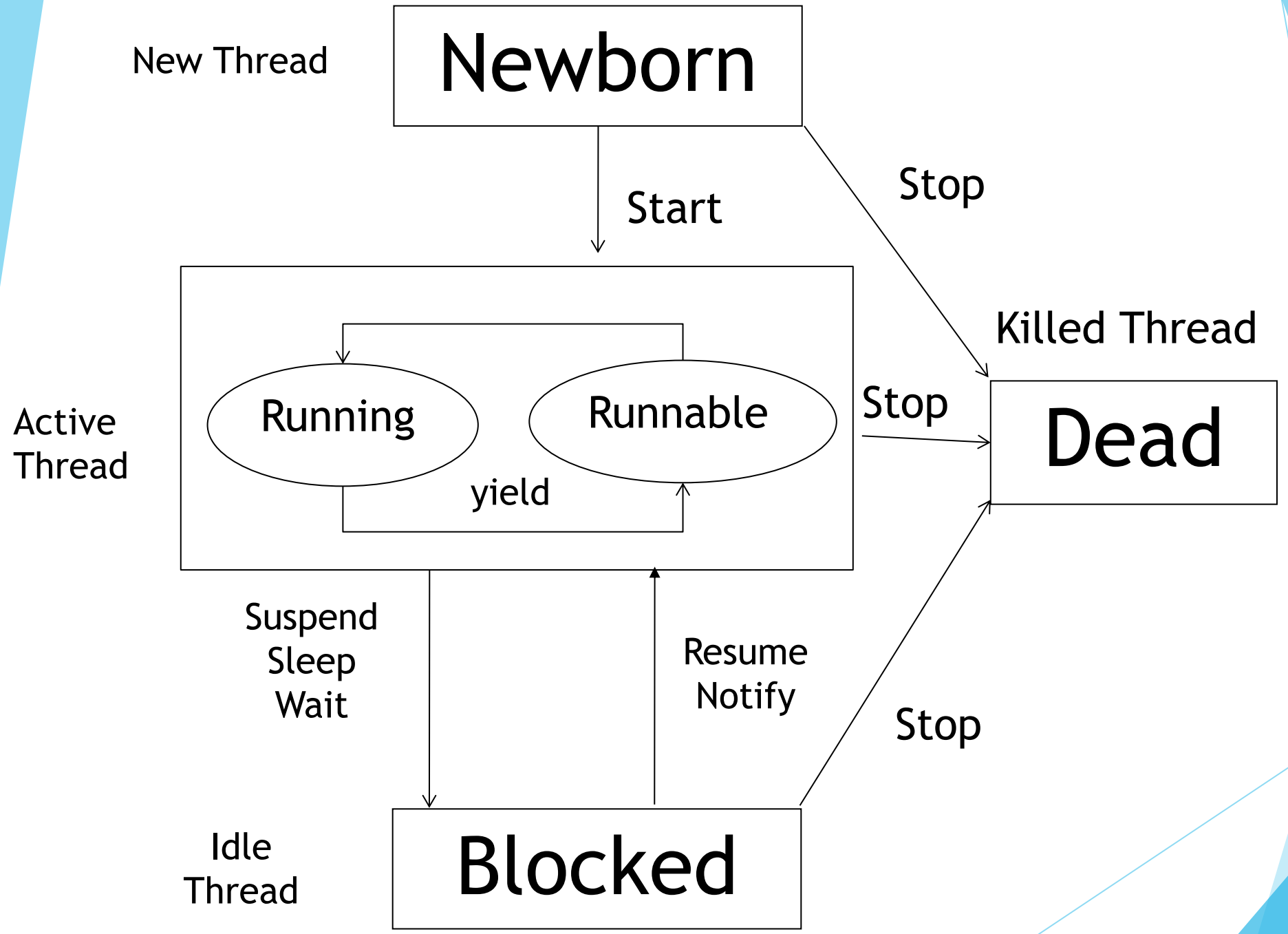
		Thread tb = new Thread(rb);
		System.out.println("Thread B Started");
		tb.start();

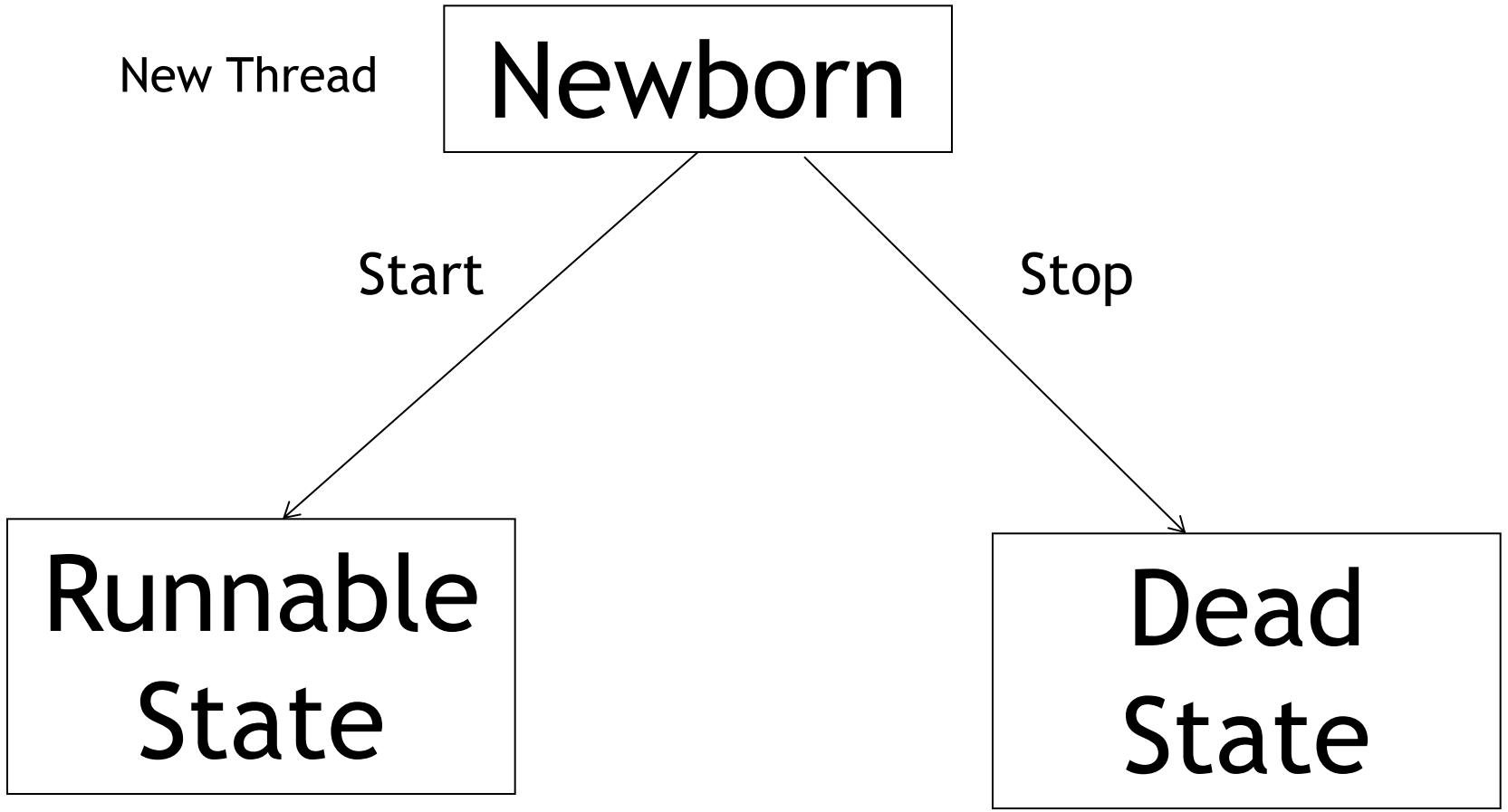
		Thread tc = new Thread(rc);
		System.out.println("Thread C Started");
		tc.start();
	}
}
```

Life Cycle of a Thread

When we are using a thread in our program. It always have a specific life cycle for all threads. In a life cycle of a thread there are 5 stages like

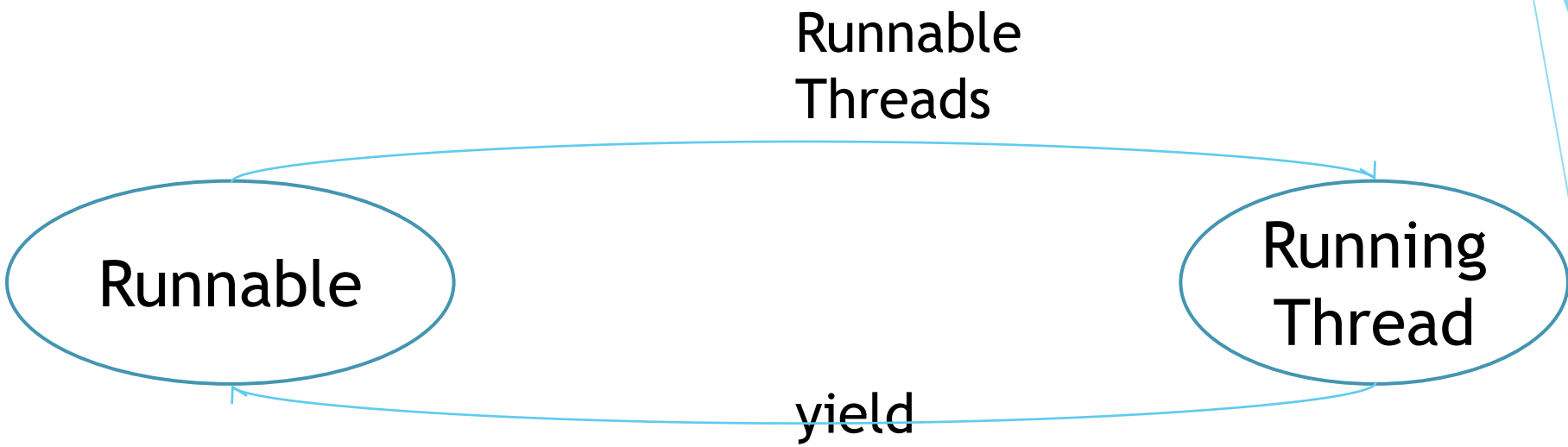
(1)Newborn state
(2) Runnable state
(3) Running state
(4) Blocked state
(5) Dead state


New Thread
Newborn
Start
Stop
Killed Thread
Active Thread
Running
Runnable
yield
Stop
Dead
Suspend
Sleep
Wait
Resume
Notify
Stop
Idle Thread
Blocked



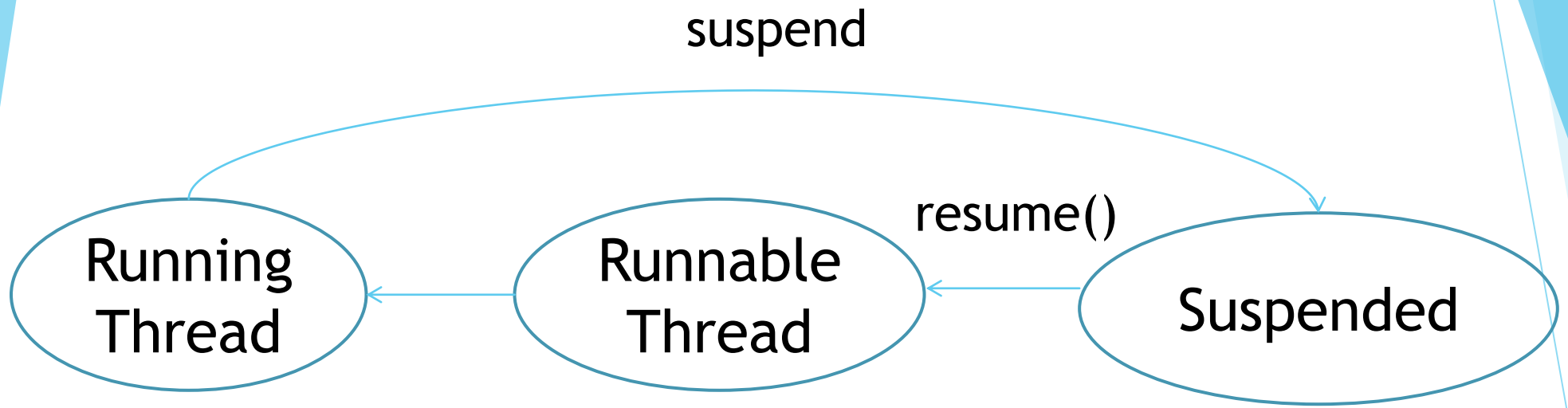
When we create any thread object, it is said to be in newborn state. At this stage we can do two things (1) start (2) stop.

Runnable State



The Runnable state means that the thread is ready for execution and is waiting for availability of the processor. The thread is in queue now and waiting for execution. Generally they will be executed in FCFS manner.

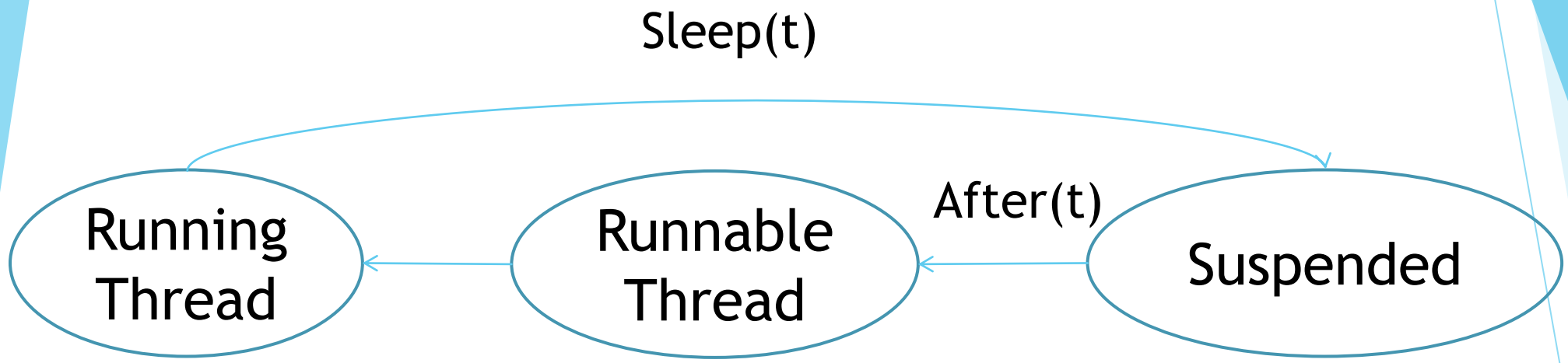
Running State



In Running state means the processor has given its time to thread for its execution. The thread runs till it gives control to another. **Suspend is deprecated method**

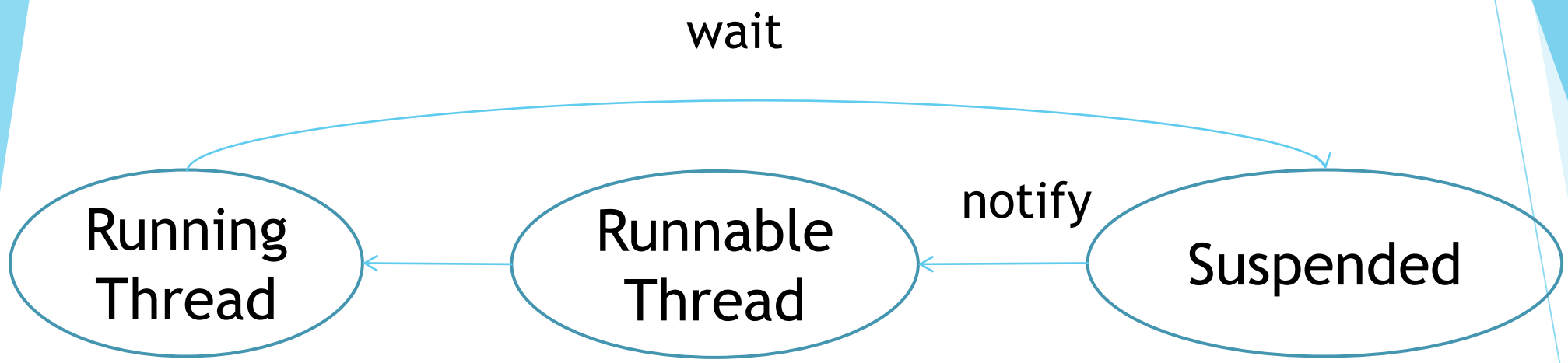
There is one method called suspend() which can be again put in running by resume() method.

Running State



We can make a thread to sleep. We can put a thread to sleep by using sleep(time) where time is in milliseconds. The thread will re-enter into the runnable state as soon as the time is over.

Running State



It has been told to wait until some event occurs. This is done using wait() method. The thread can be scheduled to run again using notify() method.

Blocked State

This is the state when the thread is still alive, but is currently not eligible to run.

Dead State

A thread is in terminated or dead state when its run() method exits.

Various Methods of Thread Class

Method	Description
void destroy()	To destroy the Thread from the memory
String getName()	Returns the name of the thread.
int getPriority()	Returns the priority of the thread.
void interrupt()	Interrupts the current thread.
boolean interrupted()	Tests whether the thread is interrupted or not
boolean isAlive()	Tests whether the thread is alive or not

Various Methods of Thread Class

Method	Description
void resume()	Resumes the thread execution
void setPriority(int newPriority)	Change the priority of the thread
void sleep(long millis)	Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
void start()	Causes this thread to begin execution; the Java Virtual Machine calls the run method of this thread.
void stop()	Stops the execution of current thread
void suspend()	Suspends the current thread execution until the resume method has been called.
void yield()	Causes the currently executing thread object to temporarily pause and allow other threads to execute.

Thread Life Cycle

```
import java.lang.*;
class a extends Thread
{
	public void run()
	{
		for(int i=1;i<=5;i++)
		{
			if(i==3)
			{
				yield();
			}
			System.out.println("From Thread A : i = "+i);
		}
		System.out.println("Exit From Thread A");
	}
}
```

Thread Life Cycle

```
class b extends Thread
{
	public void run()
	{
		for(int j=1;j<=5;j++)
		{
			System.out.println("From Thread B : j = "+j);
			if(j==3)
			{
				stop();
			}
		}
		System.out.println("Exit From Thread B");
	}
}
```

Thread Life Cycle

```
class c extends Thread{
        public void run()       {
                for(int k=1;k<=5;k++)                   {
                        System.out.println("From Thread c : k = "+k);
                        if(k==2)                                {
                                try                                     {
                                        sleep(5000);
                                }
                                catch(Exception e)
                                {}
                        }
                }
                System.out.println("Exit From Thread C");
        }
}
```

Thread Life Cycle

```
class ThreadLifeCycle
{
	public static void main(String args[])
	{
		a Thread_A = new a();
		b Thread_B = new b();
		c Thread_C = new c();

		System.out.println("Start Thread A");
		Thread_A.start();
		System.out.println("Start Thread B");
		Thread_B.start();
		System.out.println("Start Thread C");
		Thread_C.start();

	}
}
```

Thread Priority

In Java each thread will be assigned a priority, which affects the order of its execution.

If the two or more threads have same priority then they have been given equal treatment by Java scheduler and they will be processed on FCFS basis.

On the other hand we – the programmers can also set the priority by using setPriority() method.

Thread Priority

In setPriority() method we have to pass integer no. or constant according to our requirement just like

threadname.setPriority(intnumber)

Thread class has sevelral constants also they are

MIN_PRIORITY = 1

NORM_PRIORITY = 5

MAX_PRIORITY = 10

Thread Priority

Remember that the default setting is NORM_PRIORITY

When java program gets a new thread with higher priority then presently running thread will move into the runnable state and new thread will come in to execution.

Thread Priority

```
class ex167 extends Thread
{
	public void run()
	{
		System.out.println("running thread name is:"+Thread.currentThread().getName());

		System.out.println("running thread priority
is:"+Thread.currentThread().getPriority());

	}
	public static void main(String args[])
	{
		ex167 m1=new ex167();
		ex167 m2=new ex167();
		m1.setPriority(Thread.MAX_PRIORITY);
		m2.setPriority(Thread.MIN_PRIORITY);

		m1.start();
		m2.start();

	}
}
```

Synchronization

Synchronization in java is the capability to control the access of multiple threads to any shared resource.

Java Synchronization is better option where we want to allow only one thread to access the shared resource.

If the multithreading does not handle properly then data corruption can also be occurred which will leads us to system failure also.

********** Bank Example **********

Solution to Synchronization

One of the solution is use of synchronized keyword. When we use a synchronized against method, it automatically acquires a lock on the object. The lock is automatically relinquished when the method completes. So only one thread has a lock at any time so only one thread will be executed in synchronization situation.

Solution to Synchronization

```
class Table
{
    void printTable(int n)
    {
        //method not synchronized
        for(int i=1;i<=5;i++)
        {
            System.out.println(n*i);
            try
            {
                Thread.sleep(400);
            }
            catch(Exception e)
            {
                System.out.println(e);
            }
        }

    }
```

```
class MyThread1 extends Thread
{
    Table t;
    MyThread1(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(5);
    }

}
```

Solution to Synchronization

```
class MyThread2 extends Thread
{
    Table t;
    MyThread2(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(100);
    }
}
```

```
class ex164
{
    public static void main(String args[])
    {
        Table obj = new Table();
        MyThread1 t1=new MyThread1(obj);
        MyThread2 t2=new MyThread2(obj);
        t1.start();
        t2.start();
    }
}
```

Solution to Synchronization

```
class Table
{
    synchronized void printTable(int n)
    {
        //method with synchronized
        for(int i=1;i<=5;i++)
        {
            System.out.println(n*i);
            try
            {
                Thread.sleep(400);
            }
            catch(Exception e)
            {
                System.out.println(e);
            }
        }

    }
```

```
class MyThread1 extends Thread
{
    Table t;
    MyThread1(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(5);
    }

}
```

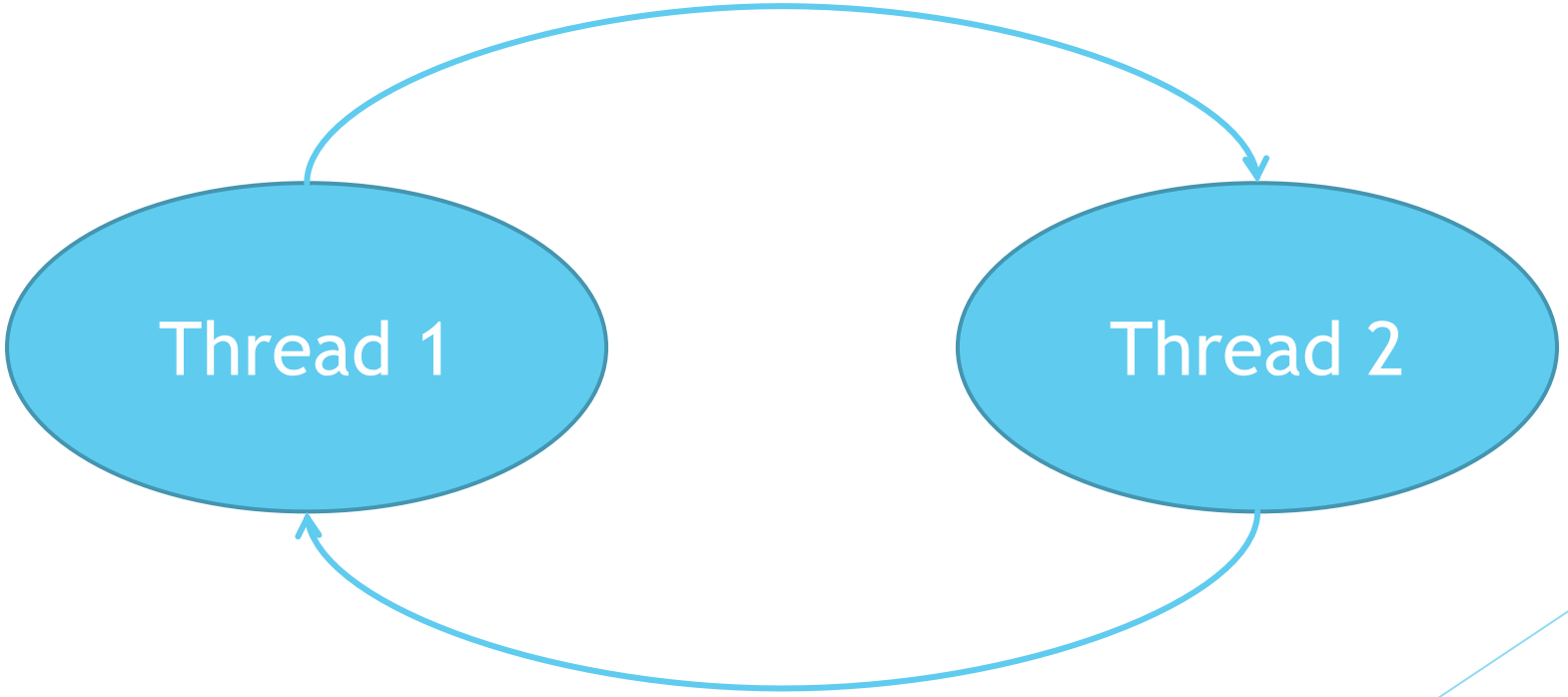
Solution to Synchronization

```
class MyThread2 extends Thread
{
    Table t;
    MyThread2(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(100);
    }
}
```

```
class ex165
{
    public static void main(String args[])
    {
        Table obj = new Table();
        MyThread1 t1=new MyThread1(obj);
        MyThread2 t2=new MyThread2(obj);
        t1.start();
        t2.start();
    }
}
```

DeadLock

Deadlock describes a situation where two or more threads are blocked forever, waiting for each other.



Grouping Threads

Every Java thread is a member of a thread group. Thread groups provide a mechanism for collecting multiple threads into a single object and manipulating those threads all at once, rather than individually. For example, you can suspend all the threads within a group with a single method call. Java thread groups are implemented by the ThreadGroup class in the java.lang package.

Need for Grouped Threads

Thread groups offer a convenient way to manage groups of threads as a unit. This is particularly valuable in situations in which you want to suspend and resume a number of related threads. For example, imagine a program in which one set of threads is used for printing a document, another set is used to display the document on the screen, and another set saves the document to a disk file. If printing is aborted, you will want an easy way to stop all threads related to printing. Thread groups offer this convenience.

The Default Thread Group

If you create a new Thread without specifying its group in the constructor, the runtime system automatically places the new thread in the same group as the thread that created it (known as the current thread group.) So, if you leave the thread group unspecified when you create your thread, the Java runtime system creates a ThreadGroup named **main**. Unless specified otherwise, all new threads that you create become members of the main thread group.

Our own Thread Group

Here we can also create our own thread group in which we can add our threads as and when we require. For creating a new thread we have ThreadGroup class and the object of this class will work as a Thread Group. Now at the time of creating the thread we have to specify this group name in the constructor of the Thread.

Various Methods of ThreadGroup

Method	Description
int activeCount()	Returns the no. of active threads in a group
int activeGroupCount()	Returns the no. of active groups in a group
void destroy()	Destroys this thread group and all of its subgroups.
String getName()	Returns the name of this thread group.
boolean isDaemon()	Tests if this thread group is a daemon thread group.
boolean isDestroyed()	Tests if this thread group has been destroyed.
void setMaxPriority(int p)	Sets the maximum priority of the group.

Common Methods we can operate on ThreadGroup

The ThreadGroup class has three methods that allow you to modify the current state of all the threads within that group:

(1) resume
(2) stop
(3) suspend

These methods apply the appropriate state change to every thread in the thread group and its subgroups.

Our own Thread Group

```
public class ex79
{
    public static void main(String[] args)
    {
        ThreadGroup grp = new ThreadGroup("group1");
        mythread m1 = new mythread(grp, "thread1");
        mythread m2 = new mythread(grp, "thread2");
        mythread m3 = new mythread(grp, "thread3");
        m1.start();
        m2.start();
        m3.start();
        grp.stop();
        System.out.println("Active Counts in Group : "+grp.activeCount());
        }
}
```

Our own Thread Group

```
class mythread extends Thread
{

	public mythread(ThreadGroup g, String s)
	{

		super(g, s);
	}
	public void run()
	{
		System.out.println("Thread name, prority, thread group " +
Thread.currentThread());
	}
}
```

Thank You